

0.87.5. La fonction `texteGrandS()` — un grand texte qui va à la ligne

`#include <texteGrandS>` : comme `texteGrand()`, mais les lettres qui ne tiennent plus repartent à gauche, une rangée de lettres plus bas.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `texteGrandS(colonne, ligne, "TEXTE", taille)` écrit un texte agrandi qui **va à la ligne** : une lettre qui dépasserait la colonne 19 repart en colonne 0, `taille` cases plus bas.

La ligne à écrire : `#include <texteGrandS>`.

Ses arguments : comme `texteGrand()`, et **tout en clair** — la place aussi : le découpage en lignes se fait avant le jeu. Un texte qui dépasserait le bas de l'écran est refusé.

Ce qu'elle coûte : environ **1 000 octets** pour deux lettres en taille 2.

Essaie : `texteGrandS(0, 0, "BONJOUR", 4);` : combien de lignes ?

On retrouve `texteGrandS` dans 1 leçon

[0.87.6](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1720 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          61 o ← la console, à l'allumage — la mise en route de la console
texte_grand_0()    146 o ← main() — ta fonction
texte_grand_1()    146 o ← main() — ta fonction
main()             26 o ← démarrage — ta fonction
VBlank             8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage      28 o ← image() — attend l'image suivante
AttendreVBlank     16 o ← démarrage — attend que l'écran soit libre
AdresseCase       10 o ← poser() — trouve l'adresse d'une case de l'écran
CopierTuiles       18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte       14 o ← démarrage — vide les deux cartes de l'écran
Tuiles            1184 o ← CopierTuiles — la police entière, et 30 tuiles dessinées
GRAND_TABLE_0_0_0  18 o ← poser() — une table gravée
GRAND_TABLE_0_1_1  18 o ← poser() — une table gravée
GRAND_TABLE_0_2_2  18 o ← poser() — une table gravée
GRAND_TABLE_1_0_3   3 o ← poser() — une table gravée
GRAND_TABLE_1_1_4   3 o ← poser() — une table gravée
GRAND_TABLE_1_2_5   3 o ← poser() — une table gravée
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre,
Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond,
EffacerPanneau, LireManette, DonneesTransfert, Notes
```

Fonctions du langage appelées :

```
poser()          × 6   182 o sur place, dans texte_grand_0(), texte_grand_1() + AdresseCase (10 o),  
GRAND_TABLE_0_0_0 (18 o), GRAND_TABLE_0_1_1 (18 o), GRAND_TABLE_0_2_2 (18 o), GRAND_TABLE_1_0_3 (3 o),  
GRAND_TABLE_1_1_4 (3 o), GRAND_TABLE_1_2_5 (3 o)
```

```
image()          × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
texte_grand_0() × 1    3 o sur place, dans main()
```

```
texte_grand_1() × 1    3 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

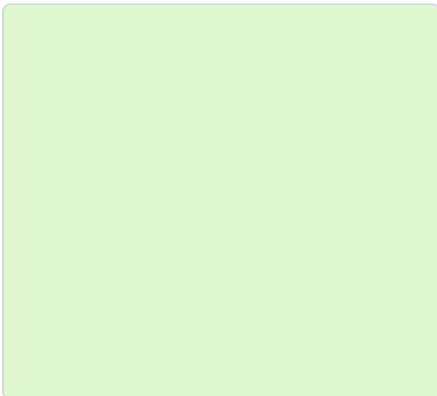
```
// ---- #include <texteGrandS> : un grand texte qui va à la ligne ----  
// En taille 3, une lettre prend 3 colonnes : 6 lettres tiennent sur 20 cases.  
// La septième, G, repart en colonne 0, trois cases plus bas.  
  
#include <texteGrandS> // texteGrandS() : agrandi, et qui va à la ligne
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

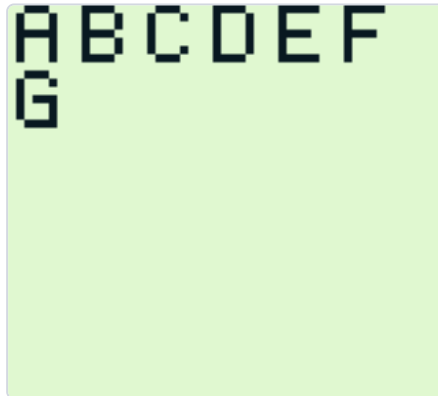
```
int main() {  
    texteGrandS(0, 0, "ABCDEFG", 3); // ABCDEF sur les lignes 0 à 2, G en (0, 3)  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

ABCDEF en grand sur toute la largeur, et G dessous, à gauche.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la première rangée de lettres est dessinée

✓ G est répartie en colonne 0, trois cases plus bas (9 cases)

Ensuite : **0.87.6. Agrandir une lettre : `texteGrand` — aller à la ligne, `texteGrandS`** — chapitre 0 : Avant tout. · [gameboy3](#) — écrire une cartouche Game Boy en C++